

Executive Summary of Project 2

-Ananya Asthana

Assignment Overview

The purpose of this assignment was to develop a solid understanding of how Remote Procedure Call (RPC) works, specifically Java Remote Method Invocation (RMI), for enabling client-server interaction. The objective was to implement a key-value store system where multiple clients could interact with a central server that processes their requests concurrently. This involved designing a server that can handle multiple client calls simultaneously and additionally ensuring the client could also send concurrent requests instead of processing them one after the other. There are many ways of implementing concurrency into our project, one method being via RMI itself which allows for RMI connection threads to be established. The scope included implementing a client server system where the server can handle multiple client calls concurrently. Additional work could also be done to allow the client to be multithreaded as well so that it could send multiple concurrent requests to the server.

Technical Impression

My experience was productive, since this assignment helped me understand why TCP and UDP no longer need to be specified when we are using RMI. Instead of manually managing sockets, RMI enables seamless remote method calls, making client-server communication more straightforward. It helped me understand how the server performs registry. Client can now directly invoke methods from my KeyValueStoreOperation implementation remotely. I faced some difficulty in implementing the thread pools on the client side so the client could send concurrent requests, as the order of my requests was getting changed due to multithreading. My requests were being processed in an unexpected order due to thread execution not being sequential. So I had to implement a solution where I used different pools of threads. This way I could clearly segregate threads assigned to a particular task. On the server side it was much simpler since I have used a ConcurrentHashMap for thread safety and RMI itself allows multithreading to be implemented by spawning threads dynamically for each client request. Another challenge I encountered was logging—since multiple threads were running in parallel, their log messages were interleaved, making it difficult to track which thread was responsible for which operation. I resolved this by modifying my log format to include thread identifiers, ensuring better visibility into concurrent operations.